LAYER 6 · PRESENTATION

L6

Encryption, Formats and Encoding

The shape and safety of the data itself: TLS encryption, JSON, TypeScript types, validating input and the bytes behind every character.

YASIR ABDULLAH · YASIR-PORTFOLIO-ZETA.VERCEL.APP/NOTES/EBOOKS/LAYER-6-PRESENTATION

SKIP TO CONTENT

Contents

1 **TLS and SSL**

How a browser and a server agree on keys, prove who the server is and encrypt everything after.

2 **JSON**

The text format almost every API speaks: its six value types and the mistakes that break it.

3 **TypeScript Types**

Catching mistakes before the code runs, and why types vanish at runtime.

4 **Input Validation**

Checking every piece of outside data on the server, against a schema, before using it.

5 **Character Encoding**

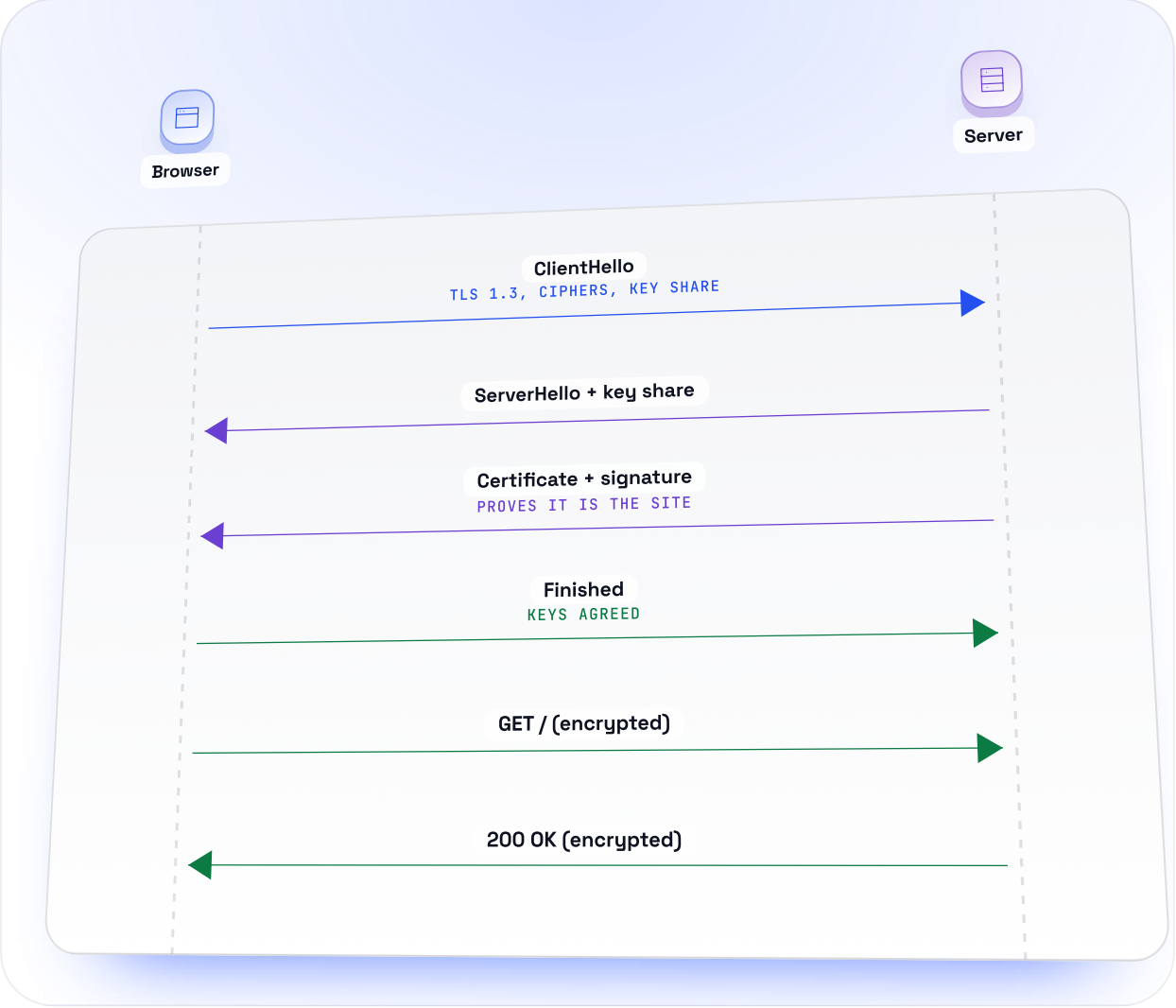
How text becomes bytes in UTF-8, and what garbled characters are telling you.

SKIP TO CONTENT

CHAPTER 1 · TLS/SSL

TLS and SSL

How a browser and a server agree on keys, prove who the server is and encrypt everything after.



TLS 1.3 in one round trip: the client offers a key share, the server answers with its own and its certificate, and both derive the same keys.

TLS (Transport Layer Security) is what puts the padlock in the address bar. It does three jobs at once: it **encrypts** the traffic, it proves the server is **who it claims to be**, and it detects any **tampering** on the way. SSL was its predecessor; every SSL version is broken and switched off, but the name stuck.

SKIP TO CONTENT

ndshake

1. **ClientHello:** the browser lists the TLS versions and ciphers it supports and sends a key share for the key exchange.
2. **ServerHello:** the server picks the version and cipher and sends its own key share. From these two shares, both sides independently compute the same secret. It never crosses the wire.
3. **Certificate and signature:** the server sends its certificate and signs the handshake with the matching private key, proving it holds it.
4. **Finished:** both sides confirm the handshake was not altered.

Everything after that, the HTTP request included, is encrypted. TLS 1.3 needs only one round trip; TLS 1.2 needed two.

Certificates and trust

A certificate binds a domain name to a public key and is signed by a **certificate authority** (CA). The browser checks:

- The name in the certificate matches the site.
- It is within its valid dates.
- It chains up, through intermediate certificates, to a root CA the device already trusts.
- It has not been revoked.

Free, automated certificates from Let's Encrypt have made HTTPS the default; most hosting platforms renew them for you.

Inspecting a site

```shell
openssl s_client -connect example.com:443 -servername example.com </dev/
curl -vI https://example.com
```

Look for the protocol version, the cipher, the certificate's subject and issuer, and `Verify return code: 0 (ok)`.

SKIP TO CONTENT

th having

- Only **TLS 1.2 and 1.3** enabled.
- **HSTS**, so browsers never try plain HTTP again:

```
TEXT

Strict-Transport-Security: max-age=31536000; includeSubDomains
```

- Redirect all HTTP to HTTPS.
- Monitor certificate expiry.

When it goes wrong

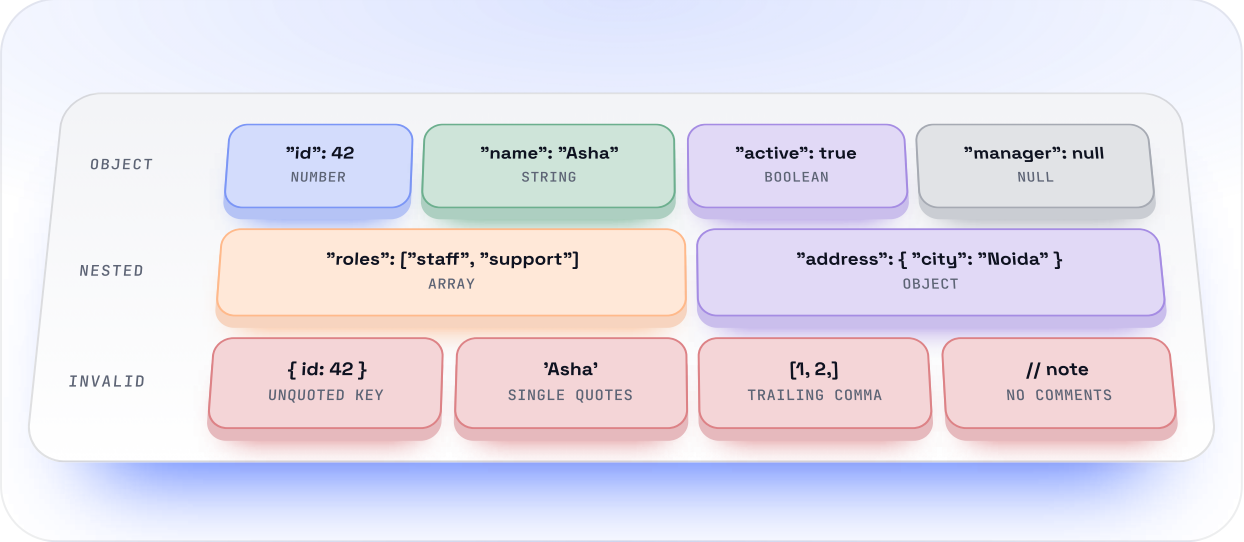
ERROR	USUAL CAUSE
Certificate expired	Renewal failed or was never automated
Name mismatch	Certificate for `example.com`, site is `www.exampl`
Unable to get local issuer	Server did not send the intermediate certificate
Works in browser, fails in an app	The app has an old trust store or pins a certific
Many sites fail on one PC	Wrong system clock, or antivirus intercepting H

SKIP TO CONTENT

CHAPTER 2 · JSON

JSON

The text format almost every API speaks: its six value types and the mistakes that break it.



One object, one value of each type. Keys are always double-quoted strings.

JSON (JavaScript Object Notation) is the text format that almost every web API sends and receives. It is small, readable by people and supported by every language. It also has strict rules, and most "invalid JSON" errors come from forgetting one of them.

Six types of value

JSON

```
{
  "id": 42,
  "name": "Asha",
  "active": true,
  "manager": null,
  "roles": ["staff", "support"],
  "address": { "city": "Noida", "pin": "201301" }
}
```

SKIP TO CONTENT

TYPE	EXAMPLE
string	`"Asha"`
number	`42`, `3.14`, `-1e3`
boolean	`true`, `false`
null	`null`
array	`["a", "b"]`
object	`{ "key": "value" }`

There is no date type. Send dates as ISO 8601 strings, `"2026-10-30T12:00:00Z"`, and parse them deliberately.

The rules people break

- Keys must be **double-quoted** strings: `{ "id": 1 }`, not `{ id: 1 }`.
- Strings use **double quotes** only.
- **No trailing commas** after the last item.
- **No comments.**
- No `undefined`, `NaN` or `Infinity`.

Reading and writing it

TS

```ts
const data = JSON.parse(text);
const text = JSON.stringify(data, null, 2);
```

`JSON.parse` throws on invalid input, so wrap it when the input comes from outside. And remember that a successful parse only proves the text is JSON, not that it has the fields you expect. Validate the shape next.

On the command line, `jq` formats and queries JSON:

SKIP TO CONTENT

```
curl -s https://api.example.com/tickets | jq '.[] | {id, status}'
```

Numbers have limits

JavaScript stores numbers as 64-bit floats, so integers above 2^53 lose precision. Send large IDs, such as database bigints and money in paise, as strings.

When it goes wrong

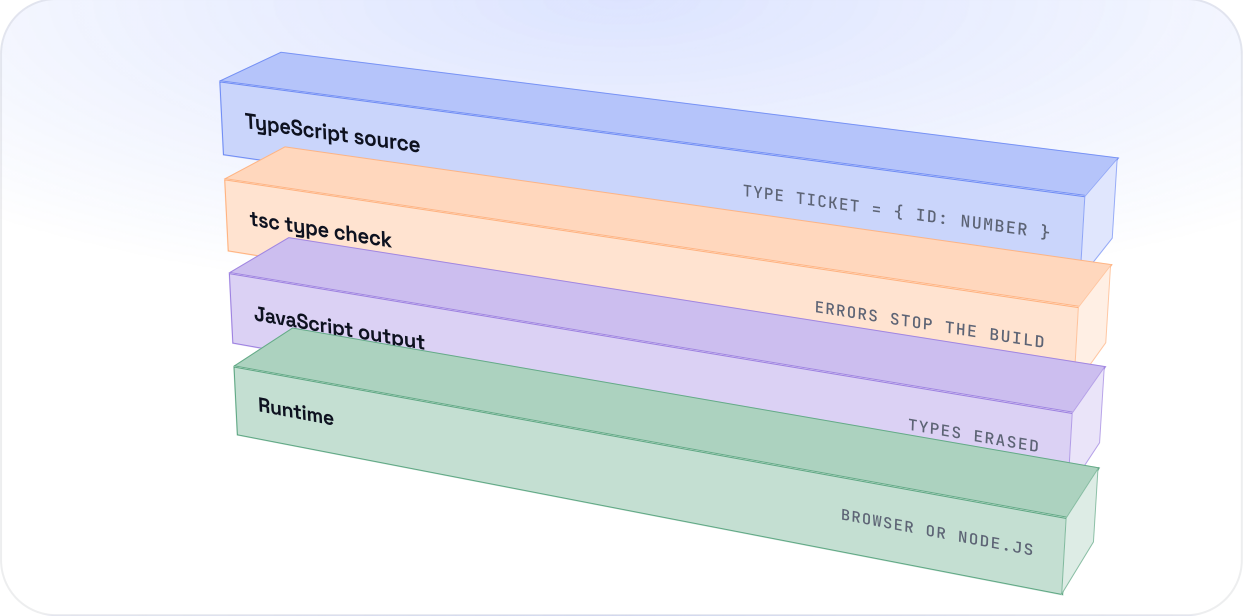
- **"Unexpected token < in JSON":** the server sent an HTML error page, not JSON. Check the status code and the response body.
- **"Unexpected end of JSON input":** the body was empty, often a 204 or a failed request.
- **IDs change by one at the end:** precision loss on a large number.
- **Garbled names:** the body was not sent or read as UTF-8.

SKIP TO CONTENT

CHAPTER 3 · TYPESCRIPT TYPES

TypeScript Types

Catching mistakes before the code runs, and why types vanish at runtime.



Types are checked when you build, then erased. The JavaScript that runs has no types, so data from outside still needs checking.

TypeScript is JavaScript with types. You describe the shape of your data, and the compiler checks every use of it before the code runs: misspelled fields, missing arguments and values that might be `undefined` are caught while you type, not by a user.

Describing data

TS

```ts
type TicketStatus = "open" | "in_progress" | "closed";

type Ticket = {
  id: number;
  title: string;
  status: TicketStatus;
  ing;
```

SKIP TO CONTENT

```
function close(ticket: Ticket): Ticket {
  return { ...ticket, status: "closed" };
}

close({ id: 1, title: "VPN down", status: "done" });
// Error: Type '"done"' is not assignable to type 'TicketStatus'.
```

- A **union** of strings (`"open" | "closed"`) is clearer and safer than a free string.
- `?` marks an optional field; TypeScript then makes you handle the missing case.

Narrowing

TypeScript follows your checks and narrows the type inside each branch:

TS

```
type Result = { ok: true; data: Ticket } | { ok: false; error: string };

function show(result: Result) {
  if (result.ok) {
    console.log(result.data.title);
  } else {
    console.error(result.error);
  }
}
```

This pattern, a **discriminated union**, makes impossible states impossible to write.

Types disappear at runtime

The compiler checks the code, then **erases** every type. The JavaScript that runs has none. So TypeScript cannot check data that arrives while the program runs: an API response, a form post, a file.

TS

```
SKIP TO CONTENT
await res.json()) as Ticket; // a promise, not a check
```

That `as` only tells the compiler to trust you. Validate outside data with a schema instead, and let the schema produce the type:

```ts
import { z } from "zod";

const TicketSchema = z.object({
  id: z.number(),
  title: z.string(),
  status: z.enum(["open", "in_progress", "closed"]),
});

type Ticket = z.infer<typeof TicketSchema>;
const ticket = TicketSchema.parse(await res.json());
```

Settings worth turning on

```json
{
  "compilerOptions": {
    "strict": true,
    "noUncheckedIndexedAccess": true
  }
}
```

`strict` turns on null checks and much more. Run `tsc --noEmit` in CI so type errors fail the build.

When it goes wrong

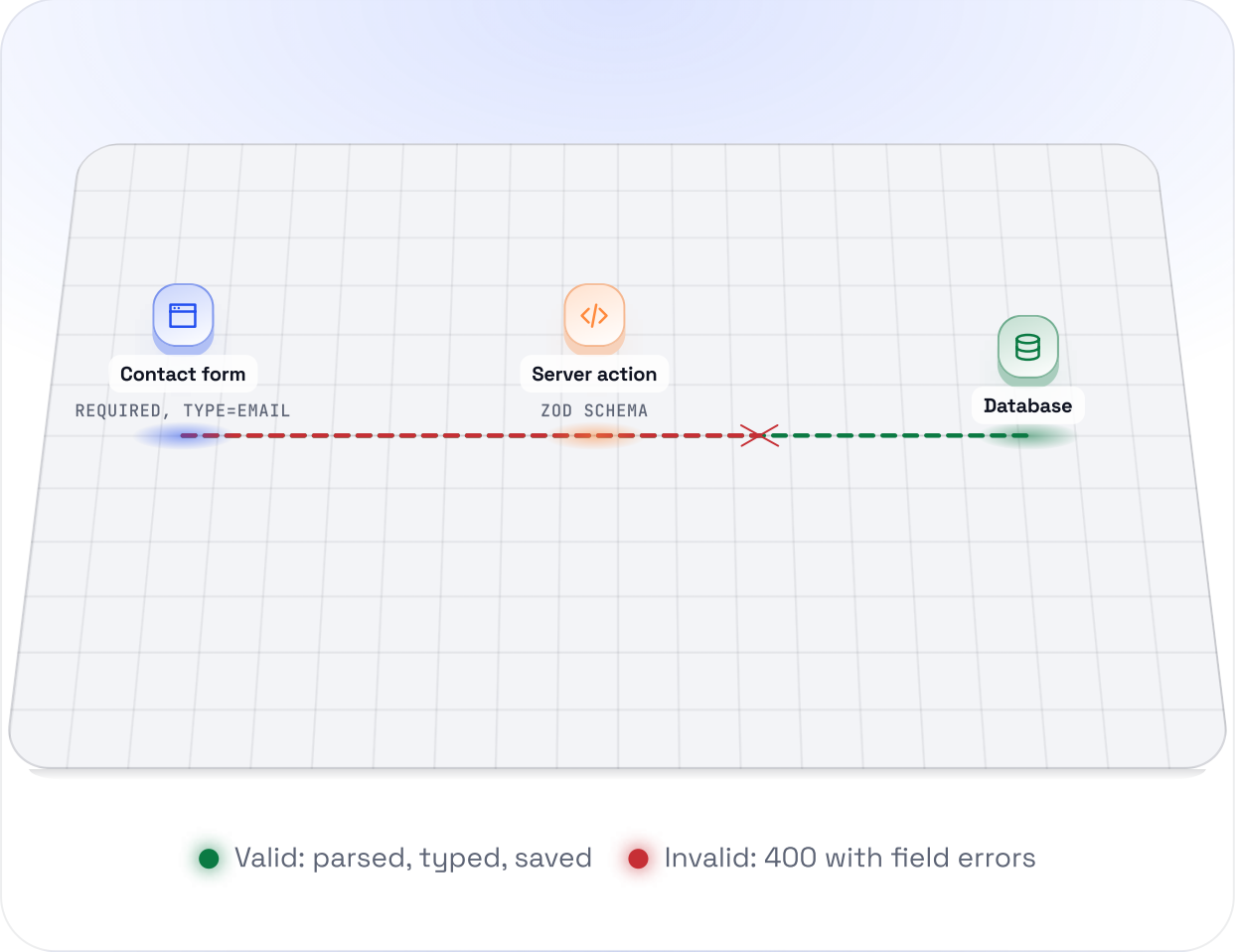
- **Types say it's a string, runtime says undefined:** the data came from outside and was cast with `as`, not validated.
- **`any` everywhere:** each `any` switches checking off. Use `unknown` and narrow it.

SKIP TO CONTENT

CHAPTER 4 · INPUT VALIDATION

Input Validation

Checking every piece of outside data on the server, against a schema, before using it.



The browser check helps the user; the server check protects the system. Bad input stops at the schema and never reaches the database.

Every piece of data that comes from outside your server, form fields, query strings, JSON bodies, headers, uploaded files, must be treated as untrusted until it is checked. Input validation is that check, and it belongs **on the server**.

Two checks, two purposes

- **In the browser:** `required`, `type="email"`, `maxlength`. These give instant ve a round trip. They are for the user's convenience.

SKIP TO CONTENT

schema that every request must pass. This is the one that protects the system, because anyone can skip the browser and send a

request directly.

Validate with a schema

Describe what valid data looks like, and reject everything else:

```ts
import { z } from "zod";

const messageSchema = z.object({
  name: z.string().trim().min(2).max(80),
  email: z.email().max(160),
  topic: z.enum(["job", "project", "other"]),
  body: z.string().trim().min(20).max(4000),
});

export async function sendMessage(formData: FormData) {
  const parsed = messageSchema.safeParse(Object.fromEntries(formData));
  if (!parsed.success) {
    return { status: "error", fieldErrors: parsed.error.flatten().fieldE
  }
  await db.message.create({ data: parsed.data });
}
```

After parsing, `parsed.data` has exactly the fields you allowed, trimmed and typed. Extra fields are dropped, so nobody can sneak `isAdmin: true` into the record.

Principles

- **Allow-list, don't block-list:** say what is allowed (an enum, a pattern, a length) rather than trying to list every bad input.
- **Check type, length, format and range.** Cap every string; an unlimited text field is an invitation.
- **Validate files by content**, not by name or the browser's MIME type: check the first bytes (the file signature) and the size.

SKIP TO CONTENT

ors per field, without echoing internals.

Validation is not the whole defence

Validation decides whether data is acceptable. You still need the right defence where the data is **used**:

RISK	DEFENCE WHERE IT IS USED
SQL injection	Parameterised queries or an ORM, never string-building
XSS	Escape on output; React does this unless you bypass it
Path traversal	Never build file paths from user input
Command injection	Never pass input to a shell

When it goes wrong

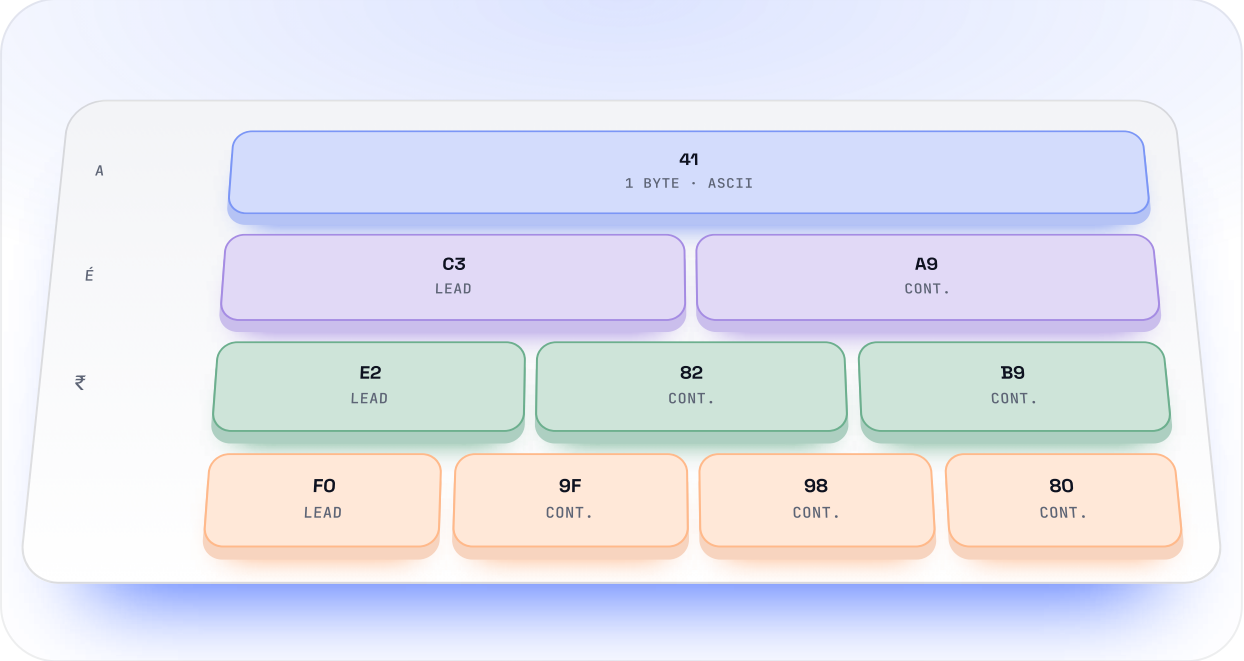
- **A field that "can't be empty" is empty in the database:** only the browser checked it.
- **A 500 error on odd input:** the server parsed without validating and crashed on the unexpected shape. It should have been a 400.

SKIP TO CONTENT

CHAPTER 5 · CHARACTER ENCODING

Character Encoding

How text becomes bytes in UTF-8, and what garbled characters are telling you.



UTF-8 uses one byte for plain English and up to four for everything else. The first byte says how many follow.

Computers store bytes, not letters. A character encoding is the rule that turns one into the other. Today that rule should almost always be **UTF-8**, and most garbled text is two programs disagreeing about it.

Characters, code points, bytes

- **Unicode** gives every character a number, its **code point**: `A` is U+0041, `é` is U+00E9, `₹` is U+20B9.
- **UTF-8** decides how each code point is written as bytes.

CHARACTER	CODE POINT	UTF-8 BYTES	BYTES
A	U+0041	41	1
SKIP TO CONTENT	U+00E9	C3 A9	2

CHARACTER	CODE POINT	UTF-8 BYTES	BYTES
₹	U+20B9	E2 82 B9	3
😜	U+1F600	F0 9F 98 80	4

Plain English text is identical in ASCII and UTF-8, which is why the problem hides until a name with an accent, a rupee sign or an emoji appears.

How UTF-8 is built

The first byte says how long the character is:

- `0xxxxxxx`: one byte (ASCII)
- `110xxxxx`: start of a 2-byte character
- `1110xxxx`: start of a 3-byte character
- `11110xxx`: start of a 4-byte character
- `10xxxxxx`: a continuation byte

Mojibake: reading the symptoms

When UTF-8 bytes are read as Windows-1252 (or Latin-1), each multi-byte character turns into several wrong ones:

YOU SEE	IT WAS	WHAT HAPPENED
`Ã©`	é	UTF-8 read as Windows-1252
`â‚¹`	₹	Same mistake, 3-byte character
``	anything	Bytes that are not valid UTF-8

Say UTF-8 everywhere

TEXT

```
tf-8" />
```

SKIP TO CONTENT

```TEXT
Content-Type: application/json; charset=utf-8
```

```TEXT
CREATE DATABASE app ENCODING 'UTF8';
```

Also: save source files as UTF-8, and in MySQL use `utf8mb4`, because its older `utf8` cannot store emoji.

Length is not what it seems

```TS
"₹".length; // 1 (UTF-16 code units)
"😀".length; // 2
new TextEncoder().encode("₹").length; // 3 bytes
```

When a database column or an API limits length, know whether it counts bytes, code units or characters.

When it goes wrong

- **CSV opens garbled in Excel:** save it as "UTF-8 with BOM", or import it and choose UTF-8.
- **Names fine on screen, broken in email:** the email was sent without a UTF-8 charset header.
- **Text cut off mid-character:** something truncated by bytes and split a multi-byte character.